

Setting Up a TWA Wrapper for LeeSide

Notes prepared with Claude — September 2026

What a TWA Is

A TWA (Trusted Web Activity) is Android's way of wrapping a website into something that installs and behaves like a real app — but instead of using a limited embedded browser view, it actually runs the live site inside real Chrome, just with the browser's address bar and toolbar hidden so it looks native. Under the hood, it's the same rendering engine and the same browsing context as a normal Chrome tab — that's the key difference from a plain “installed to home screen” PWA.

What Building It Actually Entails

- A thin Android app project — Google's Bubblewrap CLI generates this automatically from LeeSide's existing manifest.json, so it's not a rewrite of anything.
- A small verification file (assetlinks.json) hosted on the LeeSide domain, proving the app and the site are the same owner.
- A signing key for the APK. No need to publish to the Play Store at all — the APK can be sideloaded directly onto a device — though a one-time \$25 Play Developer account would allow proper distribution later if ever wanted.
- The app's actual code doesn't change at all. This is a separate, mostly one-time setup step, not an ongoing second codebase to maintain.

What It Would Resolve

1. The OAuth white-flash issue

A standalone, home-screen-installed PWA is a more restrictive browsing context than a real browser tab — Google's sign-in popup flow doesn't reliably work inside it, which is the root cause of the flash/fallback-navigation behavior that's shown up repeatedly. Since a TWA genuinely runs inside Chrome itself, it has the same session context as a normal tab, so sign-in should behave exactly like it does in a browser — no flash.

2. The predictive-back-gesture ceiling

Android's newer swipe-back gesture needs an app to declare, ahead of time, that it's going to handle back navigation — a real native capability. A web page reacting to popstate is fundamentally after-the-fact, so it can't fully participate no matter how it's coded. A TWA is a real (if thin) native app, so it can register for this properly.

What it won't touch: this is Android-only. It does nothing for the iPhone situation — iOS has no equivalent, and whatever comes up there stays exactly as separate a problem as it's always been.

Would It Affect Beta Testers?

No. A TWA is just a thin Android wrapper pointing at the live site — it doesn't touch or fork the code at all. Testers keep using the same PWA at the same URL, completely unchanged. The only new thing on the website side is the one small `assetlinks.json` verification file, which does nothing for anyone browsing normally — it's invisible to existing testers.

Step-by-Step, on the PC

1. Install Node.js

Version 14.15.0 or later.

2. Install JDK 17 specifically

Not JDK 8, not JDK 11 — Bubblewrap needs that exact major version.

3. Install the Bubblewrap CLI

```
npm i -g @bubblewrap/cli
```

4. Create a folder and initialize the project

Point it at LeeSide's manifest. The first run may offer to auto-install the Android SDK/build tools — let it, that's expected and normal.

```
bubblewrap init --manifest https://chnevill.github.io/LeeSide/manifest.json
```

5. Answer its prompts

6. Set up `assetlinks.json`

Bubblewrap needs the app's signing certificate fingerprint to generate the right content for this file. The exact command for pulling that from the keystore depends on what Bubblewrap names things during `init` — work through this together when the time comes. The resulting file gets hosted at `chnevill.github.io/LeeSide/.well-known/assetlinks.json`

7. Build the APK

This produces a signed APK file.

```
bubblewrap build
```

8. Install it on the Pixel

Copy the file over and open it (Android will ask to allow installing from that source once), or use ADB for a repeatable dev loop.

Prompts to expect during *bubblewrap init*:

- **Application ID** — a reverse-domain package name, e.g. com.chneville.leeside
- **App name, display mode (standalone), theme/background colors** — it suggests sensible defaults from the manifest
- **Notifications / shortcuts** — no, unless wanted
- **Signing key** — let it generate a new one, and set a password

Use the *same* password for both the keystore and the key alias — mismatched ones are a common source of build headaches later.

Save this password somewhere safe — it's needed for every future build, and losing it means starting over rather than being able to update this app install.

When ready to actually do this, it's worth running through steps 1–5 together, or trying them solo first and coming back if anything snags — particularly step 6 (assetlinks.json), which depends on exact output from the init step.